Week 3: Branching Strategy and SCM Plan — MediAssist

Name: Navy Lokai **Admission No.:** 672415

USIU-Africa | November 2026 | Repository: https://github.com/NavyLokai/mediassist-scm

Part 1 — OSS Branching-Policy Summary: Kubernetes

Branching model. Kubernetes uses a fork-and-pull, trunk-based model. Contributors fork kubernetes/kubernetes, clone the fork, add the official repository as an `upstream` remote and disable pushing to it, so code can only enter through a pull request (PR). All feature work branches from the default branch (`master`) on short-lived topic branches that are kept current with `git fetch upstream` and `git rebase upstream/master`; `git pull` is discouraged because merge commits break the linear history. Release branches (`release-1.x`) are cut at the `vX.Y-rc.0` release, roughly three times a year, and supported for about 12 months; after that, fixes reach them only as cherry-picks from master approved by Release Managers. Late in each cycle a Code Freeze (around week 12) admits only critical bug fixes.

Merge and review requirements. A PR can merge only when (1) the author has signed the CNCF CLA, (2) all required tests pass (non-members need a member's `/ok-to-test`), (3) a reviewer listed in the directory's `OWNERS` file adds `/lgtm`, and (4) an approver from `OWNERS` adds `/approve`. Nobody presses a merge button: the Prow bot Tide places PRs with both labels and green tests into a merge pool, re-tests them against the latest master and merges automatically; `/hold` or a WIP title blocks this. Contributors must keep PRs small and focused, split repository-wide changes per SIG, write imperative commit subjects under 50 characters, push review fixes as new commits and squash before merge, and revert through a normal PR.

Lesson for MediAssist: *adopt the parts that scale down (protected trunk, short-lived rebased branches, small PRs, automated checks as a merge gate, review recorded on the PR, tag-based releases) and drop what only a 3,000-contributor project needs (forks, OWNERS hierarchy, release branches, code freeze).*

Part 2 — SCM Plan

1. Purpose

This plan defines how every change to MediAssist, an Android medication-reminder app for elderly users with a Firebase backend and caregiver dashboard, is identified, proposed, reviewed, built, released and recorded. Because a faulty reminder can cause a missed or double dose, the plan's goals are: main is always releasable, every change is traceable from issue to installed build, and any release can be rebuilt or rolled back. It implements requirements SCM-01 to SCM-13 (Week 1) and the artifact-tracking database (Week 2).

2. Scope

Covers all configuration items in the repository: Android source (Java/Kotlin), layouts and resources, Gradle build files and the dependency lock, Firebase rules and Cloud Functions, schema migrations, tests, CI workflows, design assets and SCM documents. It covers the dev, staging and production environments and the Play Store internal, closed and production tracks. Out of scope: Firebase console settings not stored as code, and personal device setups.

3. Roles and Responsibilities

The project currently has one developer, so several roles are held by one person; the roles are kept separate so that collaborators can be added without changing the process.

Role	Holder and responsibilities
Project Owner / Configuration Manager	Navy Lokai. Owns this plan, branch-protection settings, labels and the project board; triages and approves change requests; creates release tags and approves production rollouts.
Developer	Navy Lokai (and future contributors). Works only on short-lived branches, links every PR to an issue, keeps PRs small, fixes failing checks before requesting review.
Reviewer	A second contributor or course peer added as a collaborator; one approving review is required on main. While no reviewer is available, the owner may merge with the admin bypass, which GitHub records on the PR, and must comment the reason. Critical-area PRs (reminder scheduling, alerts, auth, data rules) wait at least 12 hours before any bypass merge.
Stakeholders	Course lecturer and elderly users/caregivers in usability testing. Raise issues and confirm acceptance of releases.

4. Version Control Strategy

Chosen model: GitHub Flow (trunk-based development with short-lived branches). There is one long-lived branch, `main`, which is protected and always releasable. All work happens on short-lived branches that merge back through a PR.

Justification against project context. Team size: with one developer, Git Flow's `develop`, `release/*` and `hotfix/*` branches would add merge overhead with no one to coordinate. **Release cadence:** a build goes to internal testers every two weeks and to production about monthly; a tag on main covers this without release branches, which Kubernetes needs only because it supports several versions at once. **Risk tolerance:** low for patient-safety code, so pure commit-to-trunk is rejected: a PR is mandatory, because it is the change-control record, it runs CI before merge, and it supports a rollback by reverting one squash commit. Short branches also follow DORA's finding that small batches lower change-failure rate.

Rule	Convention
Branch names	`<type>/<issue#>-<short-description>` e.g. `feature/12-snooze-reminder`, `fix/18-duplicate-alert`. Types: feature, fix, hotfix, docs, test, chore, ci.
Branch lifetime	At most 2 working days and about 400 changed lines; larger work is split or hidden behind a feature flag so main stays releasable. Rebase on main before opening the PR (`git pull --rebase origin main`).
Commits	Conventional Commits: `feat:`, `fix:`, `docs:`, `test:`, `chore:`. Imperative subject of 50 characters or fewer; the body explains why.
Merging	Squash-merge only, so there is one commit per PR on main and a linear history. The branch is deleted automatically after merge.
Protection on main	No direct pushes or force-pushes. Requires a PR with 1 approving review, a passing `ci` check, an up-to-date branch, signed commits, linear history and resolved conversations.
Signing and rebasing	Every commit is signed with an SSH signing key registered on GitHub (shows Verified). Before review, rebase on the latest main (`git fetch origin && git rebase origin/main`) and push with `git push --force-with-lease`.

Repository layout (as pushed to GitHub):

Path	Purpose
`main`	The only long-lived branch; protected; every commit is a squash-merged PR.
`docs/`	SCM deliverables: Week 1 requirements, Week 2 database design, this plan.
`app/`	Android client source (added as development starts).
`CONTRIBUTING.md`	A short version of sections 4 to 7 for contributors.

Path	Purpose
`.github/`	PR template, issue templates (bug, feature, change request), CODEOWNERS, Dependabot config, and the ci.yml and release.yml workflows.
`CHANGELOG.md`	Release history in Keep-a-Changelog format.

5. Change Control Strategy

Every change starts as a GitHub Issue (the change request) and is classified by the Configuration Manager when triaged:

- **Minor** (docs, typos, styling): a PR with passing CI; self-merge allowed.
- **Standard** (features, non-critical fixes): an issue with acceptance criteria, then a PR with passing CI, the checklist completed and one approval (or a recorded admin bypass).
- **Critical-area** (reminder scheduling, caregiver alerts, authentication, Firebase rules, dependency upgrades): the issue must contain an impact analysis (affected items, test plan, rollback). The PR needs a second reviewer when one is available, otherwise the 12-hour wait, plus a manual test on a physical device.
- **Emergency** (production defect): a `hotfix/<issue#>-…` branch from main, a minimal fix, an expedited PR with CI still required, a PATCH tag, and a review afterwards recorded on the issue.

Your first pull request, step by step:

1. Open an issue with the right template (or pick one from *Ready*), assign yourself, and move it to *In progress*.
2. Update main: `git switch main`, then `git pull --rebase origin main`.
3. Create your branch: `git switch -c feature/<issue#>-<short-desc>`.
4. Make small commits: `git commit -m "feat: add snooze button to reminder alert"`.
5. Rebase and push: `git fetch origin && git rebase origin/main`, then `git push --force-with-lease -u origin feature/<issue#>-<short-desc>`.
6. Open the PR against `main`, fill in the template (what changed, why, how it was tested, risk area), and write `Closes #<issue#>`.
7. Check the commit shows **Verified**, wait for the `ci` check to pass, and request a review (one approval needed); for a critical-area change, also test on a physical device.
8. Click **Squash and merge**. The issue closes and the branch is deleted automatically; move on to a new branch for the next change.

6. CI/CD Process Control Strategy

GitHub Actions is the only path to a build artifact; no APK or AAB is built for release on a laptop.

- `.github/workflows/ci.yml` **— runs on every PR and push to main:** repository-structure check, Gradle lint and unit tests, debug APK build, dependency review (blocks new high or critical CVEs), and secret scanning. Its `ci` status is required before merging.
- `.github/workflows/release.yml` **— runs on a tag** `v*.*.*`: builds the signed release AAB (keystore held in GitHub Secrets), generates a CycloneDX SBOM, attaches both to a GitHub Release, and uploads to the Play internal track. Promotion to production is a manual approval step in the protected `production` environment.
- Failing checks block the merge, and failures on main are fixed or reverted within one working day. Dependabot opens weekly dependency-update PRs that follow the same rules.

7. Issue Tracking Tool and Workflow

Tool: GitHub Issues with a GitHub Projects board. Three templates (Bug report, Feature request, Change request) force the required fields. Labels: `type:*` (bug, feature, change, docs), `priority:*` (P1 to P3), `area:*` (reminders, alerts, ui, backend, ci) and `critical-area`.

Workflow: Backlog → Ready (triaged, classified and with acceptance criteria) → In progress (branch created) → In review (PR open) → Done (PR merged and issue closed by `Closes #n`). Every PR must reference an issue and every commit on main maps to one PR, which keeps the issue → PR → commit → build chain from the Week 2 database complete.

8. Release Strategy

- **Versioning:** Semantic Versioning `MAJOR.MINOR.PATCH` with an annotated tag on main (`git tag -a v1.2.0 -m "…"`). Android `versionCode` increases on every CI release build.
- **Cadence:** a fortnightly internal-track build for testers, a monthly production release, and PATCH releases on demand for hotfixes.
- **Release checklist:** all release issues are closed, CI on main is green, `CHANGELOG.md` is updated from merged PRs, accessibility checks pass (18 pt text, 4.5:1 contrast, 48 dp targets), and there are no open critical CVEs in the SBOM.
- **Rollout and rollback:** internal → closed beta (caregivers) → staged production at 10%, then 50%, then 100% over three days while monitoring crash and missed-alert reports. Rollback means halting the staged rollout and shipping the previous tag's code as a new PATCH version, because Android does not allow a lower versionCode.

References: Kubernetes Contributors, “GitHub Workflow” and “Pull Request Process”, kubernetes.dev/docs/guide; Kubernetes, “Kubernetes Release Cycle”, kubernetes.io/releases/release; DORA, Accelerate State of DevOps Report 2024, Google Cloud.